Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 1er. Semestre 2026

SIMULADOR EPIDEMIOLÓGICO

202307764 – Julio Eduardo Gustavo Alvarado Ramirez

Resumen

El presente ensayo describe el desarrollo de un simulador epidemiológico computacional basado en autómatas celulares, diseñado para investigar la expansión de enfermedades infecciosas en tejidos humanos. Este tema cobra crucial vigencia a nivel nacional en Guatemala e internacional, ya que la capacidad de predecir si una enfermedad producirá una enfermedad leve, grave o mortal resulta vital para la toma de decisiones en salud pública. A nivel técnico, el ensayo expone la adopción de una postura rigurosa basada en la Programación Orientada a Objetos (POO) y la implementación manual de Tipos de Datos Abstractos (TDA) utilizando listas enlazadas en lenguaje C#. Se omite el uso de estructuras nativas, lo cual genera un impacto técnico significativo al garantizar un control absoluto de la memoria optimizando el procesamiento de matrices masivas. Finalmente, se concluye que la gestión manual de memoria y el modelado algorítmico mediante firmas de estado permiten identificar con precisión los patrones de contagio cíclicos, demostrando que las estructuras propias son eficientes para modelar predicciones críticas.

Palabras Clave

1. Simulador epidemiológico
2. Tipos de Datos Abstractos
3. Programación Orientada a Objetos,
4. Autómatas celulares
5. C#

Abstract

This essay describes the development of a computational epidemiological simulator based on cellular automata, designed to investigate the spread of infectious diseases in human tissues. This topic gains crucial relevance at the national level in Guatemala and internationally, as the ability to predict whether a disease will produce a mild, severe, or fatal outcome is vital for public health decision-making. On a technical level, the essay presents the adoption of a rigorous approach based on Object-Oriented Programming (OOP) and the manual implementation of Abstract Data Types (ADT) using linked lists in the C# language. The use of native structures is omitted, which generates a significant technical impact by ensuring absolute memory control, optimizing the processing of massive matrices. Finally, it is concluded that manual memory management and algorithmic modeling through state signatures allow for the precise identification of cyclical contagion patterns, demonstrating that custom structures are efficient for modeling critical predictions.

Keywords

1. *Epidemiological simulator*
2. *Abstract Data Types,*
3. *Object-Oriented Programming,*
4. *Cellular automata,*
5. *C#..*

Introducción

La investigación epidemiológica busca predecir el impacto de enfermedades infecciosas a nivel celular. Computacionalmente, este comportamiento se modela mediante autómatas celulares en rejillas bidimensionales, donde la salud de una célula depende de sus vecinas. Identificar patrones cíclicos permite determinar si un paciente desarrollará un cuadro leve, grave o mortal. El propósito de este ensayo es documentar la arquitectura de software en C# desarrollada para este fin. El aporte central es demostrar cómo la implementación estricta de Programación Orientada a Objetos (POO) y Tipos de Datos Abstractos (TDA) manuales, prescindiendo de estructuras nativas, garantiza un control absoluto de la memoria al procesar matrices masivas. Para guiar la lectura, se plantean estas interrogantes: ¿Cómo optimizar la memoria sin herramientas automatizadas? y ¿Cuál es el algoritmo más eficiente para detectar ciclos biológicos exactos? Las respuestas estructurarán este documento.

Fundamentos de la Simulación Epidemiológica y Autómatas Celulares

El modelado de sistemas biológicos complejos requiere enfoques matemáticos y computacionales rigurosos. Para abordar la expansión de enfermedades infecciosas en tejidos humanos, este proyecto adopta un modelo basado en autómatas celulares. El entorno de evaluación consiste en rejillas cuadradas que representan el tejido del paciente, donde cada celda individual alberga una célula en estado saludable o contagiada. - El motor de simulación opera bajo principios deterministas y periódicos. La supervivencia o propagación del virus depende estrictamente del entorno inmediato de cada unidad celular: una célula contagiada mantiene su estado únicamente si interactúa con 2 o 3 células vecinas infectadas. Por el contrario, la infección se expande hacia una célula sana si esta se encuentra rodeada por exactamente 3 vecinas contagiadas. Este enfoque teórico permite observar comportamientos emergentes complejos a partir de reglas simples. (Ilachinski, 2001)

Gestión Manual de Memoria mediante Tipos de Datos Abstractos (TDA)

Desde una perspectiva disciplinar, el desarrollo de la solución exige un riguroso control de los recursos del sistema. El diseño arquitectónico plantea una restricción crítica: la prohibición absoluta de utilizar estructuras de datos nativas del lenguaje C#, tales como LinkedList, List, Queue o Stack. Adicionalmente, se restringe el uso de clases genéricas, forzando un paradigma de abstracción pura. (Troelsen, 2022)

Frente a esta situación, se asume una postura técnica de gestión manual de memoria mediante la implementación de Tipos de Datos Abstractos (TDA) propios. Se diseñaron clases de nodos fuertemente tipados (NodoPaciente, NodoCelda) vinculados a listas enlazadas específicas (ListaPacientes, ListaCeldas). Contraponiendo esta postura al uso de librerías automatizadas, se evidencia la ventaja de poseer un control determinista sobre los punteros de memoria, asegurando que la manipulación de las rejillas dinámicas en el paradigma de Programación Orientada a Objetos (POO) se ejecute sin la sobrecarga de procesos opacos del recolector de basura del lenguaje. (McMillan, 2007)

Algoritmo de Detección de Patrones y Evaluación de Gravedad

El propósito fundamental de la simulación es catalogar el nivel de riesgo de la enfermedad. Los científicos evalúan los datos tras "N" períodos buscando repeticiones en los patrones de infección. Si el patrón inicial reaparece consistentemente después de "N" períodos, la enfermedad se clasifica como grave. De manera crítica, si este ciclo de repetición ocurre en un intervalo de N=1, la enfermedad se considera mortal por ser incurable. (Microsoft, 2023)

Existen escenarios donde la infección muta hacia un patrón distinto a la inicial, el cual eventualmente entra en un ciclo de repetición cada N_1 períodos. Para resolver este desafío computacional de manera óptima, el algoritmo implementado extrae una "firma" de estado (una representación en cadena de texto de las coordenadas activas) en cada período. Esto permite comparar el estado actual con el historial completo de la simulación en tiempo $O(n)$, identificando los valores de N y N_1 sin la necesidad de procesar comparaciones profundas entre múltiples matrices tridimensionales. (Gansner, 2000)

Interoperabilidad de Archivos y Visualización Gráfica

La usabilidad del sistema depende de su capacidad para interactuar con repositorios de datos externos. El sistema utiliza archivos XML como insumos primarios para definir la lógica inicial, extrayendo el tamaño de la rejilla ("M"), el número de períodos y las coordenadas iniciales de la infección.

Posterior a la ejecución automática, los resultados predictivos se estructuran y exportan en un nuevo archivo XML, documentando la clasificación de la enfermedad y los períodos cíclicos detectados. Finalmente, la abstracción de los datos en memoria se materializa visualmente integrando la herramienta Graphviz. El sistema traduce las listas enlazadas dinámicas en sintaxis de tablas estructurales, permitiendo al investigador observar gráficamente la evolución celular paso a paso.

Conclusiones

El desarrollo de este simulador epidemiológico permite consolidar una postura técnica clara: la implementación manual de Tipos de Datos Abstractos (TDA), prescindiendo de las estructuras nativas y genéricas que ofrecen los lenguajes modernos, es un ejercicio fundamental que garantiza un control granular sobre la gestión de la memoria computacional. Aunque el uso de herramientas automatizadas acelera el desarrollo, la construcción de listas enlazadas fuertemente tipadas demuestra ser una arquitectura robusta y altamente eficiente para procesar el estado de miles de células simultáneamente bajo el paradigma de la Programación Orientada a Objetos (POO).

Como principal aporte algorítmico, se concluye que la técnica de "firmas de estado" —la serialización de las coordenadas activas en cadenas de texto— es una propuesta óptima para el análisis de historiales en autómatas celulares. Este enfoque minimiza la carga computacional y permite identificar con precisión matemática los períodos de repetición (*N* y *N_1*), clasificando exitosamente si el comportamiento de una enfermedad infecciosa derivará en un cuadro leve, grave o mortal para el paciente.

Finalmente, este proyecto abre el debate hacia futuras optimizaciones en el campo de la biocomputación. Quedan planteadas las siguientes interrogantes para la reflexión: ¿Cómo impactaría el uso de programación concurrente (hilos) al evaluar los vecinos de una matriz de 10,000 por 10,000 celdas? Y, si el modelo actual evalúa tejidos bidimensionales, ¿qué adaptaciones requerirían nuestras estructuras de datos manuales para simular la expansión de un virus en un entorno tridimensional y volumétrico más apegado a la realidad anatómica humana?

Bibliografía

Gansner, E. R. (2000). En E. R. Gansner, *An open graph visualization system and its applications to software engineering. Software: Practice and Experience, 30(11), 1203-1233.*

Ilachinski, A. (2001). En *Cellular automata: A discrete universe. World Scientific Publishing Company.*

McMillan, M. (2007). En M. McMillan, *Data structures and algorithms using C#. Cambridge University Press.*

Microsoft. (2023). *Microsoft*. Obtenido de Microsoft: https://learn.microsoft.com/en-us/dotnet/standard/data/xml/xml-document-object-model-dom

Troelsen, A. &. (2022). En A. &. Troelsen, *Pro C# 10 with .NET 6: Foundational principles and practices in programming. Apress.*

Anexos.

Diagrama de Clases

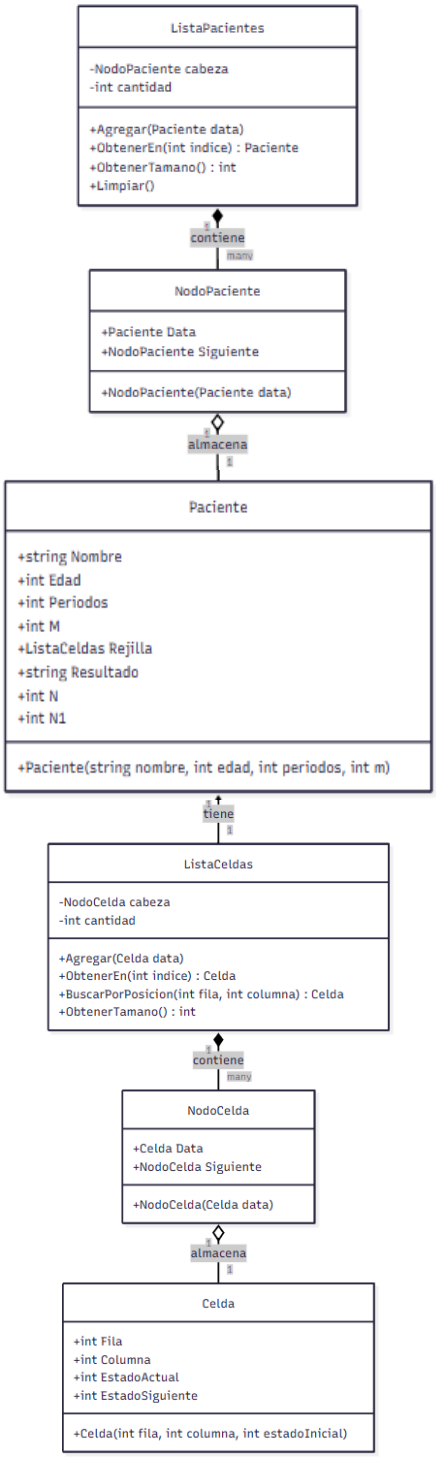

Figura 1. Diagrama de clases.
Fuente: Elaboración Propia

Diagrama de actividades

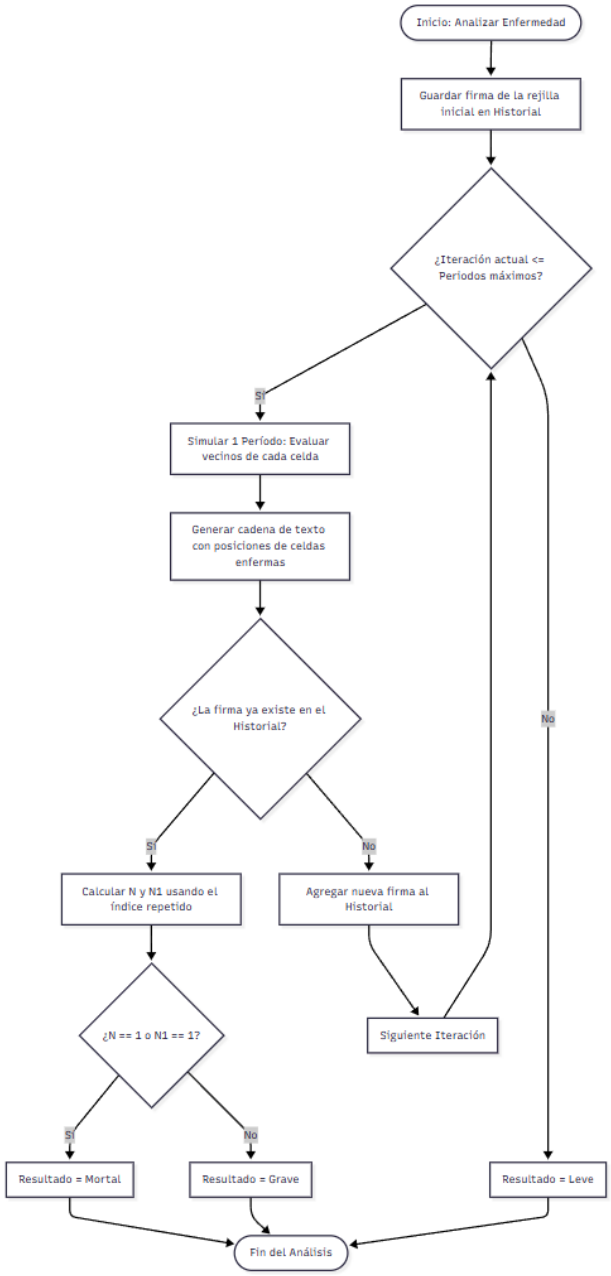

Figura 2. Diagrama de actividades
Fuente: Elaboración Propia